

Тема 9 Использование SQL с другими языками программирования высокого уровня

Чтобы вложить SQL в другой язык, вы должны использовать пакет программ, который бы обеспечивал поддержку вложения SQL в этот язык и, конечно же, поддержку самого языка. Естественно, вы должны быть знакомы с языком, который вы используете. Главным образом, вы будете использовать команды SQL для работы в таблицах базы данных, передачи результатов вывода в программу и получения ввода из программы, в которую они вкладываются, обобщенно ссылаясь к главной программе (которая может или не может принимать их из диалога или посылать обратно в диалог пользователя и программы).

Основные логические конструкции ветвления и циклов — используемые для структур большинства сред программирования, здесь отсутствуют, так что вы не сможете принять решение — выполнять ли, как выполнять, или как долго выполнять одно действие в результате другого действия. Кроме того, интерактивный SQL не может делать многого со значениями, кроме ввода их в таблицу, размещения или распределения их с помощью запросов, и конечно вывода их на какое-то устройство.

Языки программирования, однако, сильны именно в этих направлениях. Они разработаны так, чтобы программист мог начинать обработку данных, и, основываясь на ее результатах, решать, делать ли это действие или другое, или же повторять действие до тех пор, пока не встретится некоторое условие, создавая логические маршруты и циклы. Значения сохраняются в переменных, которые могут использоваться и изменяться с помощью любого числа команд. Это дает вам возможность указывать пользователям на ввод или вывод этих команд из файла, и возможность форматировать вывод сложными способами (например, преобразовывать числовые данные в диаграммы).

Цель вложенного SQL состоит в том, чтобы объединить эти возможности, позволяющие вам создавать сложные процедурные программы, которые адресуют базу данных посредством SQL, позволяя устранить сложные действия в таблицах на процедурном языке, который не ориентирован на такую структуру данных, в тоже время поддерживая структурную строгость процедурного языка.

Как делаются вложения SQL

Команды SQL помещаются в исходный текст программы, которой предшествует фраза – EXEC SQL (EXECute SQL). Далее устанавливаются некоторые команды, которые являются специальными для вложенной формы SQL и которые будут представлены в этой теме.

Строго говоря, стандарт ANSI не поддерживает встроенный SQL как таковой. Он поддерживает понятие, называемое – модуль, который более точно, является вызываемым набором процедур SQL, а не вложением в другой язык. Официальное определение синтаксиса вложения SQL, будет включать расширение официального синтаксиса каждого языка, в который может вкладываться SQL. Однако ANSI обеспечивает приложения (не являющиеся частью стандарта), которые определяют синтаксис вложения SQL для Языка C и других языков. Когда вы вставляете команды SQL в текст программы,

написанной на другом языке, надо выполнить предкомпиляцию прежде, чем окончательно ее скомпилируете.

Программа, называемая прекомпилятором (или препроцессором), будет просматривать текст программы и преобразовывать команды SQL в форму, удобную для использования базовым языком. Затем используется обычный транслятор, чтобы преобразовывать программу из исходного текста в выполняемый код.

Согласно подходу к модульному языку определенному ANSI, основная программа вызывает процедуры SQL. Процедуры выбирают параметры из главной программы и возвращают уже обработанные значения обратно в основную программу. Модуль может содержать любое число процедур, каждая из которых состоит из одиночной команды SQL. Идея в том, чтобы процедуры могли работать тем же самым способом, что и процедуры на языке, в который они были вложены (хотя модуль еще должен идентифицировать базовый язык из-за различий в типах данных различных языков).

Реализации могут удовлетворить стандарту, выполнив вложение SQL таким способом, как если бы модули уже были точно определены. Для этой цели прекомпилятор будет создавать модуль, называемый модулем доступа. Только один модуль, содержащий любое число процедур SQL, может существовать для данной программы. Размещение операторов SQL непосредственно в главном коде происходит более просто и более практично, чем непосредственно создание самих модулей.

Каждая из программ, использующих вложение SQL, связана с ID доступа во время ее выполнения. ID доступа, связанный с программой, должен иметь все привилегии, чтобы выполнять операции SQL, выполняемые в программе.

Использование переменных основного языка в SQL

Основной способ, которым SQL и части базового языка ваших программ будут связываться друг с другом – это с помощью значений переменных.

Вы можете использовать переменные из главной программы во вложенных операторах SQL везде, где вы будете использовать выражения значений.

Текущим значением переменной, может быть значение, используемое в команде. Главные переменные должны:

- быть объявленными в SQL DECLARE SESSION (РАЗДЕЛ ОБЪЯВЛЕНИЙ), который будет описан далее;

- иметь совместимый тип данных с их функциями в команде SQL (например, числовой тип, если они вставляется в числовое поле);

- должны иметь назначенное значение во время их использования в команде SQL, если команда SQL самостоятельно не может сделать назначение;

- им должно предшествовать двоеточие, когда они упоминаются в команде SQL

Так как главные переменные отличаются от имен столбцов SQL наличием у них двоеточия, вы можете использовать переменные с теми же самыми именами, что и ваши столбцы, если это конечно нужно.

Предположим, что имеетя четыре переменных в программе, с именами: `id_num`, `salesperson`, `loc` и `comm`. Они содержат значения, которые надо вставить в таблицу Продавцы. Можно вложить следующую команду SQL в программу:

```
EXEC SQL INSERT INTO Salespeople  
VALUES (:id_num, :salesperson, :loc, :comm)
```

Текущие значения этих переменных будут помещены в таблицу. Как вы можете видеть, переменная `comm` имеет то же самое имя, что и столбец, в который это значение вкладывается.

Обратите внимание, что точка с запятой в конце команды отсутствует. Это потому, что соответствующее завершение для вложенной команды SQL зависит от языка, для которого делается вложение.

Объявление переменных

Все переменные, на которые имеется ссылка в предложениях SQL, должны сначала быть объявлены в SQL DECLARE SECTION (в разделе объявлений), использующем обычный синтаксис главного языка. Вы можете иметь любое число таких разделов в программе, и они могут размещаться где-нибудь в коде перед используемой переменной, подчиненной ограничениям, определенным в соответствии с главным языком. Раздел объявлений должен начинаться и кончаться вложенными командами SQL — BEGIN DECLARE SECTION (Начало Раздела Объявлений) и END DECLARE SECTION (Конец Раздела Объявлений), которым предшествует, как обычно EXEC SQL (Выполнить).

Чтобы объявить переменные, используемые в предыдущем примере, можно ввести следующее:

```
EXEC SQL BEGIN DECLARE SECTION;  
объявление переменных  
EXEC SQL END DECLARE SECTION;
```

Кроме помещения значений переменных в таблицы, используя команды SQL, можете использовать SQL, чтобы получать значения для этих переменных. Один из способов делать это – с помощью разновидности команды SELECT, которая содержит предложение INTO. Вернемся к нашему предыдущему примеру и переместим строку Peel из таблицы Продавцов в переменные главного языка.

```
EXEC SQL SELECT snum, sname, city, comm  
INTO :id_num, :salesperson, :loc, :comm  
FROM Salespeople  
WHERE snum = 1001;
```

Выбранные значения помещаются в переменные с упорядоченными именами указанными в предложении INTO. Разумеется, переменные с именами, указанными в

предложении INTO, должны иметь соответствующий тип, чтобы принять эти значения, и должна быть своя переменная для каждого выбранного столбца.

Если не учитывать присутствие предложения INTO, то этот запрос похож на любой другой. Однако предложение INTO добавляет значительное ограничение к запросу. Запрос должен извлекать не более одной строки. Если он извлекает много строк, все они не могут быть вставлены одновременно в одну и ту же переменную. Команда естественно потерпит неудачу. По этой причине, SELECT INTO должно использоваться только при следующих условиях:

- когда используется предикат, проверяющий значения, которые могут быть уникальны, как в этом примере. Значения, которые могут быть уникальными — это те значения, которые имеют принудительное ограничение уникальности или уникальный индекс;
- когда вы используете одну или более агрегатных функций и не используете GROUP BY
- когда используете SELECT DISTINCT во внешнем ключе с предикатом, ссылающимся на единственное значение родительского ключа (обеспечивая системе предписание справочной целостности), как в следующем примере:

```
EXEC SQL SELECT DISTINCT snum  
INTO :salesnum  
FROM Customers  
WHERE snum = (SELECT snum  
              FROM Salespeople  
              WHERE sname = 'Motika');
```

Предполагалось, что Salespeople.sname и Salespeople.snum — это соответственно, уникальный и первичный ключи этой таблицы, а Customers.snum — это внешний ключ, ссылающийся на Salespeople.snum, и вы предполагали, что этот запрос произведет единственную строку.

Имеются другие случаи, когда запрос должен произвести единственную строку вывода, но они малоизвестны и, в большинстве случаев, вы основываетесь на том, что ваши данные имеют целостность, которая не может быть предписана с помощью ограничений. Программа, которая, вероятно, будет использоваться в течение некоторого времени, и лучше всего проиграть ее, чтобы быть гарантированным в будущем от возможных отказов. Во всяком случае, нет необходимости группировать запросы, которые производят одиночные строки, поскольку SELECT INTO используется только для удобства.

Как вы увидите, вы можете использовать запросы, выводящие многочисленные строки, используя курсор.

Курсор

Одно из сильных качеств SQL — это способность функционировать на всех строках таблицы, чтобы встретить определенное условие как блок-запись, не зная, сколько таких строк там может быть. Если десять строк удовлетворяют предикату, то запрос может вывести все десять строк. Если десять миллионов строк определены, все десять миллионов строк будут выведены. Это несколько затруднительно, когда вы попытаете

связать это с другими языками. Как вы сможете назначать вывод запроса для переменных, когда вы не знаете, как велик будет вывод? Решение состоит в том, чтобы использовать то, что называется курсором.

Можно рассматривать SQL курсор как устройство, которое отмечает ваше место в выводе запроса, хотя аналогия не полная.

Курсор – это вид переменной, которая связана с запросом. Значением этой переменной может быть каждая строка, которая выводится при запросе. Подобно главным переменным, курсоры должны быть объявлены прежде, чем они будут использованы. Это делается командой DECLARE CURSOR, следующим образом:

```
EXEC SQL DECLARE CURSOR Londonsales FOR  
SELECT *  
FROM Salespeople  
WHERE city = 'London';
```

Запрос не выполнится немедленно; он – только определяется. Курсор немного напоминает представление, в котором курсор содержит запрос, а содержание курсора напоминает любой вывод запроса, каждый раз, когда курсор становится открытым. Однако, в отличие от базовых таблиц или представлений, строки курсора упорядочены: имеются первая, вторая... ..и последняя строка курсора. Этот порядок может быть произвольным, с явным управлением с помощью предложения ORDER BY в запросе или же по умолчанию следовать какому-то упорядочению, определяемому инструментальной схемой.

Когда вы находите точку в вашей программе, в которой вы хотите выполнить запрос, вы открываете курсор с помощью следующей команды:

```
EXEC SQL OPEN CURSOR Londonsales;
```

Значения в курсоре могут быть получены, когда вы выполняете именно эту команду, но не предыдущую команду DECLARE и не последующую команду FETCH. Затем, вы используете команду FETCH, чтобы извлечь вывод из этого запроса, по одной строке в каждый момент времени.

```
EXEC SQL FETCH Londonsales INTO :id_num, :salesperson,  
:loc, :comm;
```

Это выражение переместит значения из первой выбранной строки, в переменные. Другая команда FETCH выведет следующий набор значений. Идея состоит в том, чтобы поместить команду FETCH внутрь цикла, так, чтобы, выбрав строку, вы могли, переместив набор значений из этой строки в переменные, возвращались обратно в цикл, чтобы переместить следующий набор значений в те же самые переменные.

Пока курсор закрыт, SQL не следит за тем, какие строки были выбраны. Если вы открываете курсор снова, запрос повторно выполняется с этой точки, и вы начинаете все сначала.

Этот пример не обеспечивает автоматический выход из цикла, когда все строки уже будут выбраны. Когда у FETCH нет больше строк, которые надо извлекать, он просто не

меняет значений в переменных предложения INTO. Следовательно, если данные исчерпались, эти переменные будут неоднократно выводиться с идентичными значениями, до тех пор, пока пользователь не завершит цикл, введя ответ – N.

SQLCODE

Надо знать о том, что, когда данные будут исчерпаны, то можно было сообщить об этом пользователю, и цикл завершился бы автоматически. Это – даже более важно чем, например, знать, что команда SQL выполнена с ошибкой. Переменная SQLCODE предназначена обеспечить эту функцию. Она должна быть определена как переменная главного языка и должна иметь тип данных, который в главном языке соответствует одному из точных числовых типов SQL, как это показано в Приложении В. Значение SQLCODE устанавливается каждый раз, когда выполняется команда SQL. В основном существуют три возможности:

1. Команда выполнилась без ошибки, но не произвела никакого действия. Для различных команд это выглядит по-разному:

- а) Для SELECT ни одна строка не выбрана запросом.
- б) Для FETCH последняя строка уже была выбрана, или ни одной строки не выбрано запросом в курсоре.
- в) Для INSERT ни одной строки не было вставлено (подразумевается, что запрос использовался, чтобы сгенерировать значения для вставки, и был отвергнут при попытке извлечения любой строки).
- г) Для UPDATE и DELETE ни одна строка не ответила условию предиката, и, следовательно, никаких изменений сделано в таблице не будет.

В любом случае, будет установлен код SQLCODE = 100.

2. Команда выполнилась нормально, не удовлетворив ни одному из выше указанных условий. В этом случае, будет установлен код SQLCODE = 0.

3. Команда сгенерировала ошибку. Если это случилось, изменения, сделанные к базе данных текущей транзакцией, будут восстановлены.

В этом случае будет установлен код SQLCODE = некоторому отрицательному числу, определяемому проектировщиком. Задача этого числа – идентифицировать проблему так точно, насколько это возможно. В принципе, программа должна быть снабжена подпрограммой, которая, в этом случае, должна выполняться, чтобы выдать для нас информацию, расшифровывающую значение негативного числа, определенного вашим проектировщиком. В этом случае некоторое сообщение об ошибке будет выведено на экран или записано в файл протокола, а программа в это время выполнит восстановление изменений для текущей транзакции, отключится от базы данных и выйдет из нее.

Предложение WHENEVER

Это удобно для выхода при выполненном условии – все строки выбраны. Но если получена ошибка, предпринять нечто такое, что описано для третьего случая, выше. Для этой цели, SQL предоставляет предложение GOTO. Фактически, SQL позволяет нам применять его достаточно широко, так что программа может выполнить команду GOTO автоматически, если будет произведено определенное значение SQLCODE. Можно сделать это совместно с предложением WHENEVER. Имеется кусок из примера для этого случая:

```
EXEC SQL WHENEVER SQLERROR GOTO Error_handler;  
EXEC SQL WHENEVER NOT FOUND CONTINUE;
```

SQLERROR – это другой способ сообщить, что $SQLCODE < 0$; а NOT FOUND – это другой способ сообщить, что $SQLCODE = 100$. (Некоторые реализации называют последний случай еще как – SQLWARNING.)

Error_handler – это имя того места в программе, в которое будет перенесено выполнение программы, если произошла ошибка. Такое место определяется любым способом, соответствующим для главного языка. Метка более удачно идентифицирует стандартную процедуру, распространяемую проектировщиком для включения во все программы.

CONTINUE не делает чего-то специального для значения SQLCODE. Оно также является значением по умолчанию, если не используется команда WHENEVER, определяющая значение SQLCODE. Однако, эти неактивные определения дают нам возможность переключаться вперед и назад, выполняя и не выполняя действия, в различных точках (метках) программы. Например, если программа включает в себя несколько команд INSERT, использующих запросы, которые реально должны производить значения, можно напечатать специальное сообщение или сделать что-то такое, что поясняло бы, что запросы возвращаются пустыми и никакие значения не были вставлены. В этом случае, можно ввести следующее:

```
EXEC SQL WHENEVER NOT FOUND GOTO No_rows;
```

No_rows – это метка в некотором коде, содержащем определенное действие. С другой стороны, если нужно сделать выборку в программе позже, можно ввести следующее в этой точке:

```
EXEC SQL WHENEVER NOT FOUND CONTINUE;
```

что бы выполнение выборки повторялось до тех пор, пока все строки не будут извлечены, что является нормальной процедурой, не требующей специальной обработки.

Модифицирование курсоров

Курсоры могут также быть использованы, чтобы выбирать группу строк из таблицы, которые могут быть затем модифицированы или удалены одна за другой. Это дает нам возможность обходить некоторые ограничения предикатов, используемых в командах UPDATE и DELETE. Можно ссылаться на таблицу, задействованную в предикате запроса курсора или любом из его подзапросов, которые нельзя выполнить в предикатах самих

этих команд. Стандарт SQL отклоняет попытку удалить всех пользователей с рейтингом ниже среднего, в следующей форме:

```
EXEC SQL DELETE FROM Customers  
WHERE rating < (SELECT AVG (rating)  
FROM Customers);
```

Однако, можно получить тот же эффект, используя запрос для выбора соответствующих строк, запомнив их в курсоре, и выполнив DELETE с использованием курсора. Сначала надо объявить курсор:

```
EXEC SQL DECLARE Belowavg CURSOR FOR  
SELECT *  
FROM Customers  
WHERE rating < (SELECT AVG (rating)  
FROM Customers);
```

Затем надо создать цикл, чтобы удалить всех заказчиков с помощью курсора:

```
EXEC SQL WHENEVER SQLERROR GOTO Error_handler;  
EXEC SQL OPEN CURSOR Belowavg;  
  
Начало цикла  
  
EXEC SOL FETCH Belowavg INTO :a, :b, :c, :d, :e;  
EXEC SOL DELETE FROM Customers  
WHERE CURRENT OF Belowavg;  
конец цикла  
  
EXEC SOL CLOSE CURSOR Belowavg;
```

Предложение WHERE CURRENT OF означает, что DELETE применяется к строке, которая в настоящее время выбрана курсором. Здесь подразумевается, что и курсор, и команда DELETE ссылаются на одну и ту же таблицу, и, следовательно, что запрос в курсоре – это не объединение.

Курсор должен также быть модифицируемым. Являясь модифицируемым, курсор должен удовлетворять тем же условиям что и представления. Кроме того, ORDER BY и UNION, которые не разрешены в представлениях, в курсорах – разрешаются, но предохраняют курсор от модифицируемости. Обратите внимание в вышеупомянутом примере, что мы должны выбирать строки из курсора в набор переменных, даже если мы не собирались использовать эти переменные. Этого требует синтаксис команды FETCH. UPDATE работает так же.

Например, можно увеличить значение комиссионных всем продавцам, которые имеют заказчиков с оценкой=300, следующим способом. Сначала надо объявить курсор:

```
EXEC SOL DECLARE CURSOR High_Cust AS  
SELECT *  
FROM Salespeople  
WHERE snum IN (SELECT snum  
FROM Customers  
WHERE rating = 300);
```

Затем выполняется модификация в цикле:


```

EXEC SQL OPEN CURSOR High_cust;
начало цикла

    EXEC SQL FETCH High_cust
    INTO :id_num, :salesperson, :loc, :comm;
    EXEC SQL UPDATE Salespeople
    SET comm = comm + .01
    WHERE CURRENT OF High_cust;
конец цикла
EXEC SQL CLOSE CURSOR High_cust;

```

Обратим внимание: что некоторые реализации требуют, чтобы мы указывали в определении курсора, что курсор будет использоваться для выполнения команды UPDATE на определенных столбцах. Это делается с помощью заключительной фразы определения курсора — FOR UPDATE <column list>. Чтобы объявить курсор High_cust таким способом, так чтобы можно было модифицировать командой UPDATE столбец comm, надо ввести следующее предложение:

```

EXEC SQL DECLARE CURSOR High_Cust AS
SELECT *
FROM Salespeople
WHERE snum IN (SELECT snum
                FROM Customers
                WHERE rating = 300)
FOR UPDATE OF comm;

```

Это обеспечит определенной защитой от случайных модификаций, которые могут разрушить весь порядок в базе данных.

Переменная INDICATOR

Пустые (NULLS) значения – это специальные маркеры определяемые самим SQL. Они не могут помещаться в главные переменные. Попытка вставить NULL значения в главную переменную будет некорректна, так как главные языки не поддерживают NULL значений из SQL, по определению. Хотя результат при попытке вставить NULL значение в главную переменную определяет проектировщик, этот результат не должен противоречить теории базы данных, и поэтому обязан произвести ошибку: код SQLCODE в виде отрицательного числа, и вызвать подпрограмму управления ошибкой. Естественно, вам нужно этого избежать. Поэтому, можно выбрать NULL значения с допустимыми значениями, не приводящими к разрушению программы. Даже если программа и не разрушится, значения в главных переменных станут неправильными, потому что они не могут иметь NULL значений. Альтернативным методом, предоставляемым для этой ситуации, является функция переменной indicator (указатель).

Переменная indicator, объявленная в разделе объявлений SQL напоминает другие переменные. Она может иметь тип главного языка, который соответствует числовому типу в SQL. Всякий раз, когда выполняется операция, которая должна поместить NULL значение в переменную главного языка, надо использовать переменную indicator, для надежности. Надо поместить переменную indicator в команду SQL непосредственно после переменной главного языка, которую надо защитить, без каких-либо пробелов или запятых, хотя можно, при желании, вставить слово INDICATOR.

Переменной `indicator` в команде изначально присваивается значение 0. Однако, если производится значение `NULL`, переменная `indicator` становится равной отрицательному числу. Вы можете проверить значение переменной `indicator`, чтобы узнать, было ли найдено значение `NULL`.

Нет ничего такого в разделе объявлений, что могло бы представить их как переменные `indicator`. Они станут переменными `indicator`, когда будут использоваться как переменные `indicator`.

Если будут обнаружены `NULL` значения, выполнится еще одна часть программы, которая распечатает предупреждающее сообщение, где было найдено каждое `NULL` значение.

Обратите внимание: переменные `indicator` должны проверяться в главном языке, как указывалось выше, а не в предложении `WHERE` команды `SQL`.

Последнее в принципе не запрещено, но результат часто бывает непредвиденным.

Использование переменной `INDICATOR` для эмуляции `NULL` значений `SQL`

Другая возможность состоит в том, чтобы обрабатывать переменную `indicator`, связывая ее с каждой переменной главного языка специальным способом, эмулирующим поведение `NULL` значений `SQL`.

Всякий раз, когда мы используем одно из этих значений в программе, например в предложении `if ... then`, вы можете сначала проверить связанную переменную `indicator`, является ли ее значение `=NULL`. Если это так, то обрабатывается переменная по-другому. Например, если `NULL` значение было извлечено из поля `city` для главной переменной `city`, которая связана с переменной `indicator` – `i_city`, надо установить значение `city`, равное последовательности пробелов. Это будет необходимо, только если надо будет распечатывать его на принтере; его значение не должно отличаться от логики программы. Естественно, `i_city` автоматически устанавливается в отрицательное значение.

Любое значение, вводимое в переменную `city`, или будет равно `'London'`, или не будет равно. Следовательно, в каждом случае значение коммиссионных будет либо увеличено, либо уменьшено. Однако эквивалентные команды в `SQL` выполняются по-разному:

```
EXEC SQL UPDATE Salespeople  
SET comm = comm + .01  
WHERE city = 'London';
```

и

```
EXEC SQL UPDATE Salespeople  
SET comm = comm - .01;  
WHERE city <> 'London';
```

Если значение `city` в варианте на `SQL` будет равно значению `NULL`, оба предиката будут неизвестны, и значение `comm`, следовательно, не будет изменено в любом случае.

В более сложной программе, можно установить Булеву переменную в "верно", чтобы указать, что значение city = NULL. Затем можно просто проверить эту переменную всякий раз, когда это необходимо.

Другое использование переменной INDICATOR

Переменная indicator также может использоваться для назначения значения NULL. Просто добавьте ее к имени главной переменной в команде UPDATE или INSERT тем же способом что и в команде SELECT. Если переменная indicator имеет отрицательное значение, значение NULL будет помещено в поле. Например, следующая команда помещает значения NULL в поля city и comm, таблицы Продавцы, всякий раз, когда переменные indicator – i_a или i_b будут отрицательными; в противном случае она помещает туда значения главных переменных:

```
EXEC SQL INSERT INTO Salespeople  
VALUES (:Id_num, :salesperson, :loc:i_a, :comm:i_b);
```

Переменная indicator используется также, чтобы показывать отбрасываемую строку. Это произойдет если вставляются значения символов SQL в главную переменную, которая не достаточно длинна чтобы вместить все символы. Это особая проблема с нестандартными типами данных – VARCHAR и LONG. В этом случае, переменная будет заполнена первыми символами строки, а последние символы будут потеряны. Если используется переменная indicator, она будет установлена в положительное значение, указывающее на длину отбрасываемой части строки, позволяя, таким образом, узнать, сколько символов было потеряно.

В этом случае, можно проверить с помощью просмотра – значение переменной indicator > 0, или < 0.